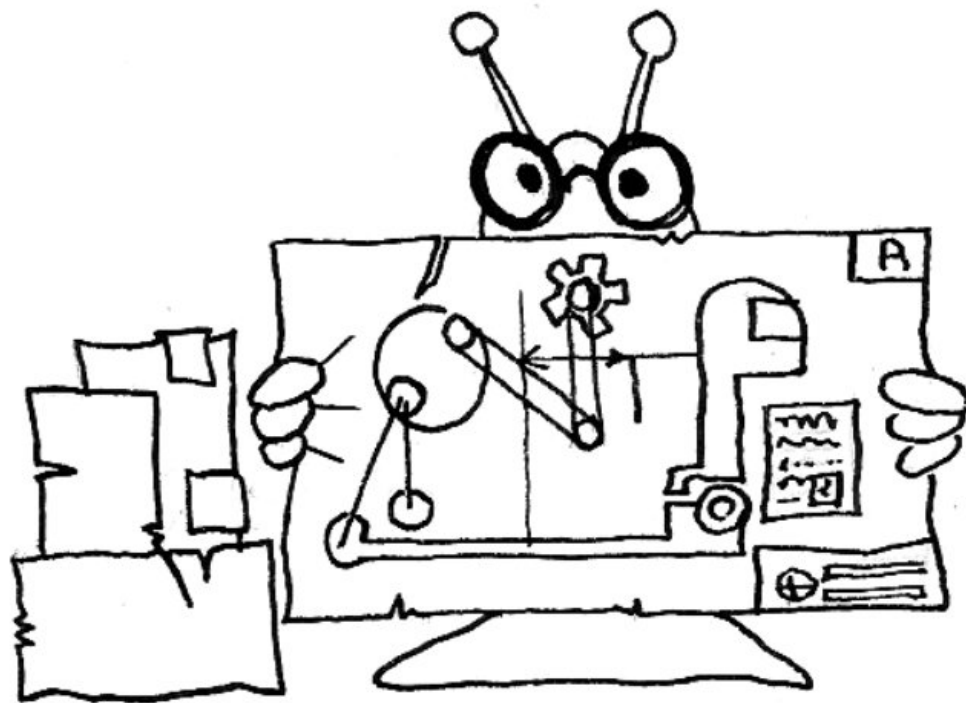


Chapitre 2: Commandes Linux Fondamentales



Objectifs du chapitre

1. Naviguer dans le système de fichiers avec les chemins absolus et relatifs
2. Gérer les fichiers et répertoires (`touch`, `cp`, `mv`, `rm`, `mkdir`) et les métacaractères
3. Consulter le contenu des fichiers (`cat`, `less`, `head`, `tail`, `wc`, `sort`, `uniq`, `cut`)
4. Rediriger les flux standard (`>`, `>>`, `<`, `2>`) et construire des pipelines (`|`)
5. Configurer l'environnement du shell (variables, `PATH`, `~/.bashrc`)
6. Archiver et compresser (`tar`, `gzip`, `bzip2`, `xz`) et localiser les fichiers (`find`, `locate`)
7. Objectif transverse : **gagner en autonomie dans le terminal**

Section 1 : Le shell et les bases de la ligne de commande

Qu'est-ce qu'un shell ? → les shells disponibles → invite et syntaxe →
internes/externes → historique et complétion → aide

Qu'est-ce qu'un shell ?

- **Shell** : interpréteur de commandes qui lit les commandes saisies par l'utilisateur et lance les programmes correspondants ; c'est l'interface principale entre l'utilisateur et le noyau sur les systèmes Unix.
- Le shell est un programme comme les autres — pas le noyau
- Il exécute une boucle : **invite** → **saisie** → **analyse** → **exécution** → **invite**
- La plupart des commandes sont des programmes **externes** qu'il lance pour vous
- Le shell par **défaut** sous Linux est **bash**

Analogie : le shell est l'« interprète » entre vous et le système

Les shells disponibles

- **bash** (Bourne Again Shell) : shell par défaut de la plupart des distributions Linux
- **sh** (Bourne shell) : l'ancêtre ; sur Ubuntu, /bin/sh pointe vers dash
- **csh** / **tcsh** (C shell) : syntaxe proche du C, populaire sur BSD
- **ksh** (Korn shell) : fusionne Bourne et C shell
- **zsh** (Z shell) : extension moderne de bash (par défaut sur macOS)

Le shell par défaut est configuré pour chaque utilisateur

Le terminal, l'invite et la syntaxe des commandes

- **Terminal (émulateur)** : fenêtre qui héberge le shell (gnome-terminal, konsole, xterm, WSL)
- **Invite (prompt)** : texte que le shell affiche en attendant une commande, ex. `ilias@pc:~$`
- **Syntaxe** : `commande` `[options]` `[arguments]`
- Les options modifient le comportement (ex. `ls -l`) ; les arguments désignent la cible
- Séparer les éléments par des espaces
- Options courtes `-l` et longues `--all` : souvent équivalentes

Exemple : disséquer `ls -la /home`

`ls` : la commande — liste le contenu d'un répertoire

`-la` : options combinées — `-l` liste longue, `-a` inclut les fichiers cachés

`/home` : l'argument — le répertoire à lister

- **Sortie** : permissions, liens, propriétaire, groupe, taille, date, nom
- Les permissions ne sont qu'un aperçu : détaillées au chapitre 07

Astuce : `man ls` révèle toutes les options

Commandes internes vs externes

- **Commande interne (builtin)** : intégrée au shell — `cd`, `pwd`, `echo`, `exit`, `type`
- **Commande externe** : programme séparé cherché dans les répertoires du `$PATH`

`type cmd` indique le type ; `type -a cmd` liste toutes les versions

- Si interne et externe existent, l'interne a **priorité**
- Exécuter un programme hors du PATH : donner son chemin (ex. `./prog`)
- Le `$PATH` sera détaillé dans la section 8

Historique, complétion et édition de ligne

- **Complétion** : touche **Tab** complète commandes et noms de fichiers ; **double Tab** affiche les choix
- **Historique** : **history** affiche les dernières commandes ; stocké dans `~/.bash_history`
- **Rappel** : flèche `↑`, `!!` (dernière commande), `!n` (commande n° n)
- **Recherche** : `Ctrl+R` recherche dans l'historique
- **Édition** : `Ctrl+A` début, `Ctrl+E` fin, `Ctrl+K` efface jusqu'à la fin, `Ctrl+W` efface le mot

Ces raccourcis réduisent les fautes de frappe

Obtenir de l'aide : man, info, --help

man **commande** : page de manuel — une référence, pas un tutoriel

- Les pages **man** sont organisées en sections : 1 commandes, 2 appels système, 3 bibliothèques, 5 formats de fichiers, ..., 8 administration
- **man -k mot-clé** : recherche par mot-clé ; **man 5 passwd** : section précise
- **info** : documentation hypertexte GNU

commande **--help** : aide rapide ; **help builtin** : aide des commandes internes

- Dans **man** : Espace page suivante, **/motif** recherche, **q** quitter (pageur less)

Section 2 : Navigation dans le système de fichiers

Chemins absolus et relatifs → pwd, cd → l'arborescence Linux (FHS) → ls → fichiers cachés

Chemins absolus vs relatifs

- **Chemin absolu** : chemin complet depuis la racine `/`, indépendant du répertoire courant (ex. `/etc/ssh/sshd_config`)
- **Chemin relatif** : chemin résolu à partir du répertoire courant (ex. `../documents/rapport.pdf`)
- Un chemin absolu commence toujours par `/`
- `..` = répertoire parent ; `.` = répertoire courant ; `~` = répertoire personnel
- **Absolu** : idéal dans les scripts ; **relatif** : rapide à taper
- `pwd` affiche le chemin absolu du répertoire courant

Le répertoire courant : `pwd`, `cd`, `~`, `.`, `..`

- `pwd` : affiche le répertoire de travail courant (Print Working Directory)
- `cd` répertoire : change de répertoire — commande interne au shell
- `cd` ou `cd ~` : retour au répertoire personnel
- `cd ..` : remonte au parent ; `cd ../..` : deux niveaux
- `cd -` : revient au répertoire précédent
- `pushd`/`popd` : pile de répertoires pour naviguer rapidement

L'arborescence Linux : la FHS (*Filesystem Hierarchy Standard*)

- **/** : racine de tout le système
- **/bin**, **/sbin** : commandes essentielles (utilisateurs / administration)
- **/etc** : fichiers de configuration du système
- **/home** : répertoires personnels des utilisateurs
- **/usr** : programmes et bibliothèques (partageable, montable en lecture seule)
- **/var** : données variables (journaux, spool, mail)
- **/tmp** : fichiers temporaires
- **/dev** : fichiers de périphériques
- **/proc**, **/sys** : fichiers virtuels du noyau
- **/mnt**, **/media** : points de montage
- **/lib**, **/opt**, **/root**, **/srv** : bibliothèques, logiciels additionnels, root, services

	Shareable	Unshareable
Static	/usr /opt	/etc /boot
Variable	/home /var/mail	/var/run /var/lock

ls : options principales

- **ls** : liste le contenu d'un répertoire, trié par nom par défaut
- **ls -l** : liste longue — permissions, propriétaire, groupe, taille, date
- **ls -a** : inclut les fichiers cachés ; **ls -A** : sans . et ..
- **ls -h** : tailles lisibles (Ko, Mo) — très souvent combiné : **ls -lh**
- **ls -R** : récursif ; **ls -S** : tri par taille ; **ls -t** : tri par date
- **ls -F** : marque les types (/ dossier, @ lien, * exécutable)

Fichiers cachés et lecture de `ls -la`

- Un fichier dont le nom commence par `.` est caché (ex. `.bashrc`, `.profile`)
- `ls -la` révèle les fichiers cachés ; les dot files servent aux fichiers de configuration personnels
- Exemple : `-rw-r--r-- 1 ilias users 1234 Jul 10 09:41 notes.txt`
 - - type · `rw-r--r--` permissions · `1` liens · `ilias` propriétaire · `users` groupe · `1234` taille ·
date · nom
- Permissions détaillées au chapitre 07 — ici, lecture des colonnes seulement
- Le répertoire personnel est désigné par `~` (ex. `/home/ilias`)

Section 3 : Gestion des fichiers et des répertoires

Types de fichiers → touch, cp → mv, rm → mkdir, rmdir → exemple combiné

Les types de fichiers

- Le premier caractère de `ls -l` indique le type de fichier
 - `-` fichier ordinaire • `d` répertoire • `l` lien symbolique
 - `c` périphérique caractère • `b` périphérique bloc • `s` socket • `p` tube nommé
 - « Tout est fichier » sous Linux, y compris les périphériques (/dev)
- `file nom` détermine le type réel : `file /etc/passwd` → « ASCII text »
- Les liens (`l`) seront étudiés en détail au chapitre 06

Créer et copier : touch, cp

- **touch** **fichier** : crée un fichier vide, ou met à jour sa date de modification
- **cp source dest** : copie un fichier ; **cp -r dossier dest** : copie récursive
- **cp -i** : demande confirmation avant d'écraser ; **cp -u** : copie si la source est plus récente
- **cp -p** : préserve permissions et dates ; **cp -a** : copie « archive » (attributs, liens)
- **Astuce** : slash final sur le dossier de destination (**cp f /dir/**) pour éviter les erreurs
- **Attention** : **cp** écrase silencieusement par défaut

Déplacer, renommer, supprimer : mv, rm

- **mv source dest** : déplace **ET** renomme — Linux ne distingue pas les deux
- **mv fichier dossier/** : déplace ; **mv a.txt b.txt** : renomme
- **mv** sur le même système de fichiers est instantané (simple réécriture des entrées)
- **rm fichier** : supprime définitivement — pas de corbeille !
- **rm -r dossier** : supprime récursivement ; **rm -i** : demande confirmation
- **Prudence extrême** : **rm -rf /** détruirait le système — vérifier avant de valider

mkdir et rmdir

- **mkdir dossier** : crée un répertoire
- **mkdir -p a/b/c** : crée toute la hiérarchie intermédiaire manquante
- **mkdir -m 700 dossier** : crée avec des permissions précises (mode octal)
- **rmdir dossier** : supprime un répertoire VIDE uniquement
- **rmdir -p a/b/c** : supprime la hiérarchie si tous les niveaux sont vides
- Pour un répertoire non vide : utiliser **rm -r**

Exemple : opérations combinées

- `cp -r project/ project_backup/` : sauvegarde complète d'un dossier
- `mkdir -p a/b/c/d` : crée une hiérarchie entière en une commande
- `rm -i temp/*.log` : suppression vérifiée des fichiers de journal
- `mv projet_ancien.pdf archive/rapports/` : déplacement vers un sous-dossier
- `touch .placeholder` : marqueur caché pour conserver un dossier vide
- **Réflexe** : `ls -la` après chaque opération pour vérifier

Section 4 : Métacaractères (globbing)

Définition → les motifs *, ?, [] → classes, intervalles, accolades → globbing vs regex

Le globbing : définition

- **Globbing (expansion des noms de fichiers)** : mécanisme par lequel le shell remplace un motif contenant des métacaractères (`*`, `?`, `[]`) par la liste des noms de fichiers correspondants, avant l'exécution de la commande.
- Les métacaractères sont **interprétés par le shell**, pas par la commande
- `ls *.txt` devient effectivement `ls fichier1.txt fichier2.txt ...`
- La commande ne reçoit que des noms concrets
- La correspondance respecte la casse : `*.TXT` $\neq$ `*.txt`

Les fichiers cachés ne sont pas sélectionnés par les motifs simples

Les motifs : *, ?, []

- ***** : n'importe quelle chaîne, y compris vide — **b*k** → **book**, **balk**, **bk**, **backtrack**
- **?** : exactement UN caractère — **b??k** → **book**, **balk**, **buck**
- **[abc]** : un caractère parmi la classe — **b[ae]ok** → **baok**, **beok**
- **[a-z]** : intervalle — **b[a-z]ck** → **back**, **buck**...
- Combinaisons libres : **photo-[0-9].jpg**
- Les motifs fonctionnent avec toutes les commandes (**ls**, **rm**, **cp**, **mv**...)

Classes, intervalles, négation et accolades

- Classes POSIX : `[:alpha:]`, `[:digit:]`, `[:lower:]`, `[:upper:]`, `[:alnum:]`
- Négation : `[!abc]` (ou `[^abc]`) : un caractère qui n'est PAS dans la classe
- Intervalles combinés : `[A-Za-z0-9]` : toute lettre ou tout chiffre
- Expansion d'accolades : `{a,b,c}` énumère — `echo {a,b}{1,2}` → `a1 a2 b1 b2`
- **Astuce** : `cp fichier{,.bak}` copie vers fichier.bak
- Les accolades sont expansées par le shell AVANT le globbing

Classes, intervalles, négation et accolades

- Classes POSIX : `[:alpha:]`, `[:digit:]`, `[:lower:]`, `[:upper:]`, `[:alnum:]`
- Négation : `[!abc]` (ou `[^abc]`) : un caractère qui n'est PAS dans la classe
- Intervalles combinés : `[A-Za-z0-9]` : toute lettre ou tout chiffre
- Expansion d'accolades : `{a,b,c}` énumère — `echo {a,b}{1,2}` → `a1 a2 b1 b2`
- **Astuce** : `cp fichier{,.bak}` copie vers fichier.bak
- Les accolades sont expansées par le shell AVANT le globbing

Quiz 1 : Shell, navigation et fichiers

- Q1. Quelle commande affiche le répertoire de travail courant ?
 - A. `cd` • B. `pwd` • C. `ls -l` • D. `whoami`
- Q2. Que répond `type pwd` ?
 - A. `/usr/bin/pwd` • B. `pwd` is a shell builtin • C. une erreur • D. la valeur du PATH
- Q3. Dans `ls *.txt`, qui interprète le motif `*.txt` ?
 - A. `ls` • B. le noyau • C. le shell • D. le terminal
- Q4. Quelle commande *copie* un dossier avec tout son contenu ?
 - A. `cp dossier` • B. `copy dossier` • C. `cp -r dossier` • D. `mv dossier`

Section 5 : Consultation du contenu des fichiers

cat, tac, less → head, tail → wc → sort, uniq → cut → exemple combiné

cat, tac et less

- **cat fichier** : affiche tout le fichier (concatenate) — utile pour les fichiers courts
- **cat f1 f2 > fusion** : concatène plusieurs fichiers en un seul
- **cat -n** : numérote les lignes
- **tac fichier** : affiche les lignes en ordre inverse
- **less fichier** : pagineur pour les fichiers longs, écran par écran
- **Astuces pour less** : Espace (page), flèches (ligne), **/motif** (recherche), q (quitter)

head, tail et tail -f

- **head fichier** : les 10 premières lignes ; head -n 20 : 20 lignes
- **tail fichier** : les 10 dernières lignes ; tail -n 50 : 50 lignes
- **tail -f fichier** : suit le fichier en direct — indispensable pour les journaux
- **head -n 15 fichier | tail -n 5** extrait les lignes 11 à 15

Ctrl+C interrompt le suivi de tail -f

- **Cibles privilégiées** : les fichiers de /var/log

wc : compter

- **wc fichier** : lignes, mots, octets — ex. 308 2343 15534 fichier
- **wc -l** : lignes uniquement ; **wc -w** : mots ; **wc -c** : octets ; **wc -m** : caractères
- **wc -L** : longueur de la ligne la plus longue
- En bout de pipeline : **cat log.txt | wc -l** compte les entrées du journal
- **wc** lit l'entrée standard si aucun fichier n'est donné

sort et uniq

- **sort fichier** : trie les lignes (ordre alphabétique ASCII par défaut)
- **sort -n** : tri numérique ; **sort -r** : ordre inverse ; **sort -f** : ignore la casse
- **sort -k2** : tri sur le champ n° 2 ; **sort -M** : tri par mois
- **uniq** : supprime les lignes DUPLIQUÉES ADJACENTES
- **uniq -c** : compte les occurrences ; **uniq -d** : uniquement les doublons
- **Combinaison classique** : **sort fichier** | **uniq -c** | **sort -rn** (fréquences)

cut et exemple combiné

- `cut -d: -f1,3 /etc/passwd` : extrait les champs 1 et 3, séparateur :
- `cut -c1-10` : les 10 premiers caractères de chaque ligne
- `cut -d " " -f 11` : séparateur espace (extraction d'un mot)
- Pipeline : `cut -d: -f1 /etc/passwd | sort | uniq` : utilisateurs uniques
- `sort -k2 -n data.txt | uniq -c | sort -rn` : analyse de fréquences
- `tail -f /var/log/syslog` : surveillance en direct (à tester au TP2)

Section 6 : Redirection

Les trois flux standard → descripteurs de fichiers → >, >> → < → 2>, 2>>, &>, 2>&1 → exemple

Les trois flux standard

- **Flux standard** : canaux d'entrée/sortie pré-connectés que chaque processus hérite à son lancement : stdin, stdout et stderr.
- **stdin (0)** : entrée standard — par défaut le clavier
- **stdout (1)** : sortie standard — les données normales, par défaut l'écran
- **stderr (2)** : sortie d'erreur — les messages d'erreur, par défaut l'écran aussi
- **Les programmes traitent ces flux comme des fichiers**
- **Pourquoi séparer stdout et stderr ?** Pour ne pas mélanger **données** et **erreurs**

Descripteurs de fichiers

- **fd 0** — stdin — entrée
- **fd 1** — stdout — sortie
- **fd 2** — stderr — erreur
- **>** redirige stdout ; **2>** redirige stderr ; **&>** redirige les deux
- **> fichier** : crée ou écrase ; **>> fichier** : ajoute
- **/dev/null** : le « trou noir » — ex. **2>/dev/null**

Descripteurs de fichiers

- `commande > fichier` : écrit stdout dans fichier — ÉCRASE le contenu existant
- `commande >> fichier` : AJOUTE stdout à la fin de fichier
- `echo "texte" > notes.txt` : crée un fichier texte en une commande

Le fichier est créé automatiquement s'il n'existe pas

- `>` et `>>` ne concernent que stdout (**fd 1**)
- **Prudence** : `>` écrase sans avertissement — préférez `>>` pour conserver

Rediriger l'entrée : <

- **commande < fichier** : utilise fichier comme entrée standard
- Exemple : **wc -l < notes.txt** compte les lignes depuis le fichier
- **sort < noms.txt** : trie le contenu du fichier
- Souvent équivalent à passer le fichier en argument (**sort noms.txt**)
- **<< MOT** : here-document — texte intégré jusqu'à la ligne MOT (utile dans les scripts)
- **tr** exige l'entrée standard : **tr a-z A-Z < fichier**

Rediriger l'erreur : 2>, 2>>, &> et 2>&1

- **commande 2> erreurs.log** : stderr vers le fichier ; **2>>** : ajoute
- **commande &> tout.log** : stdout + stderr dans le même fichier
- **commande > f.log 2>&1** : redirige d'abord stdout, puis copie stderr vers stdout
- L'ordre compte : **2>&1 > f.log** ne fusionne pas comme attendu !
- **2>/dev/null** : masque les erreurs (ex. accès refusés)
- Séparer **> données.txt 2> erreurs.log** : données propres, erreurs isolées

Exemple : combiner les redirections

- `ls /tmp > listing.txt` : sauvegarde le listing d'un dossier
- `find /etc -type f > fichiers.txt 2> erreurs.log` : séparer données et erreurs
- `ls /nonexistent 2>&1 | less` : voir l'erreur dans le pagineur
- `echo "note" >> journal.log` : ajouter au lieu d'écraser
- **Vérification** : `cat fichiers.txt` et `cat erreurs.log`
- Question piège : si rien n'échoue, erreurs.log est vide — c'est normal

Section 7 : Pipes

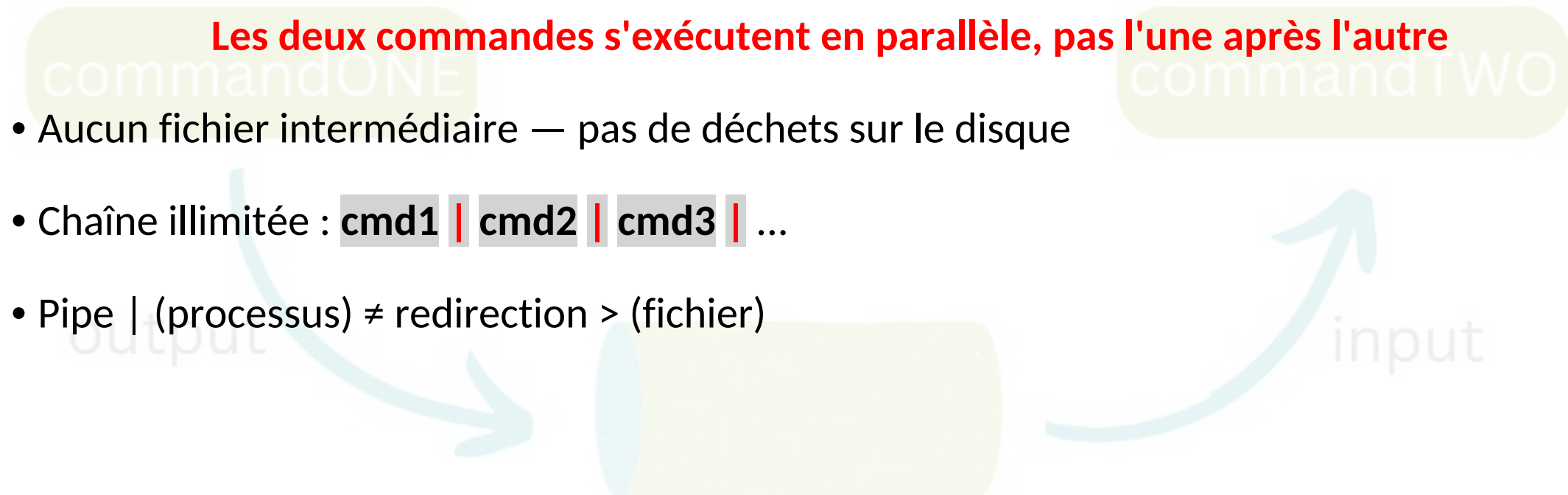
Le pipe : définition → pipeline → philosophie Unix → tee → exemple

Exemple : combiner les redirections

- **Pipe** : mécanisme qui connecte la sortie standard d'une commande à l'entrée standard d'une autre, noté `|` ; il transporte un flux d'octets unidirectionnel entre deux processus.
- `cmd1 | cmd2` : le stdout de `cmd1` devient le stdin de `cmd2`

Les deux commandes s'exécutent en parallèle, pas l'une après l'autre

- Aucun fichier intermédiaire — pas de déchets sur le disque
- Chaîne illimitée : `cmd1 | cmd2 | cmd3 | ...`
- Pipe `|` (processus) $\neq$ redirection `>` (fichier)



La philosophie Unix

- « Faire une seule chose, et la faire bien » (Doug McIlroy)
- Des outils simples et spécialisés, combinables à l'infini
- La sortie de l'un = l'entrée de l'autre : le texte comme format universel
- ls, sort, head, wc, cut... de petits programmes au rôle unique
- Composer des outils plutôt qu'écrire des programmes monolithiques
- Linux hérite directement de cette philosophie

tee : voir et sauvegarder

- **commande | tee fichier** : affiche la sortie ET l'écrit dans un fichier
- **tee -a fichier** : ajoute au lieu d'écraser
- Utile pour journaliser tout en surveillant en direct
- Exemple : **echo "hello world" | tee output.txt | wc -w**
- Le terminal **affiche** le texte ET **output.txt** le contient

Exemple : construire un pipeline

- `tail -n 100 /var/log/syslog | sort | uniq -c | sort -rn` : entrées les plus fréquentes
- `echo "hello world" | tee output.txt | wc -w` : compte et sauvegarde
- `cut -d: -f1 /etc/passwd | sort | uniq -c | sort -rn` : fréquence des comptes
- `ps aux | sort -k3 -rn | head` : processus les plus gourmands (aperçu CM04)
- Lire un pipeline : de gauche à droite, chaque étape transforme
- Ces pipelines seront construits au TP2

Quiz 2 : Redirections et pipes

- Q1. `commande > f.txt 2>&1` : où se retrouve la sortie d'erreur ?
 - A. à l'écran • B. dans f.txt • C. dans un fichier 2.txt • D. elle est perdue
- Q2. `commande 2>&1 > f.txt` : où se retrouve la sortie d'erreur ?
 - A. à l'écran • B. dans f.txt • C. dans les deux • D. elle est perdue
- Q3. Que fait `tee` dans `echo "hello" | tee sortie.txt` ?
 - A. supprime la sortie • B. affiche ET enregistre • C. trie la sortie • D. compte les mots
- Q4. `head -n 15 f.txt | tail -n 5` affiche...
 - A. les 15 premières lignes • B. les lignes 11 à 15 • C. les 5 dernières lignes • D. tout le fichier

Section 8 : Variables d'environnement et configuration du shell

Définition → variables courantes → PATH → export, echo, env
→ fichiers de configuration → shell de login vs non-login

Variable d'environnement : définition

- **Variable d'environnement** : valeur nommée stockée dans le bloc d'environnement d'un processus, héritée par ses processus enfants, qui configure le comportement des programmes.
- **Convention** : noms en **MAJUSCULES** (PATH, HOME, USER...)
- **Lire** : `echo $VAR` — le `$` précède le nom
- **Affecter** : `VAR=valeur` ; transmettre : `export VAR` (ou `export VAR=valeur`)

Héritée par les processus enfants (sous-shells, programmes lancés)

- **Liste complète** : `env` ou `printenv`

Les variables courantes

- **PATH** : répertoires de recherche des commandes
- **HOME** : répertoire personnel • **USER** / **LOGNAME** : nom d'utilisateur
- **SHELL** : chemin du shell courant
- **PS1** : format de l'invite • **EDITOR** : éditeur préféré des programmes
- **LANG** : langue/locale • **TERM** : type de terminal
- **PWD** : répertoire courant • **HOSTNAME** : nom de la machine • **TZ** : fuseau horaire

PATH : résolution des commandes

- **PATH** : liste de répertoires séparés par **:**, parcourus dans l'ordre
- **echo \$PATH** → ex. **/usr/local/bin:/usr/bin:/bin:/usr/games**
- Taper **ls** → le shell cherche **ls** dans chaque répertoire du **PATH**
- Aucun résultat : **command not found**
- Ajouter son dossier : **export PATH="\$HOME/bin:\$PATH"** dans ~/.bashrc
- **Sécurité** : ne jamais placer **.** (dossier courant) dans le PATH, surtout pour root

export, echo \$VAR, env, printenv

- **echo \$PATH** : affiche la valeur d'une variable
- **MA_VAR=42** : variable locale au shell — non transmise aux enfants
- **export MA_VAR=42** : variable d'environnement — transmise
- **env** : liste toutes les variables ; **env | sort** : liste triée
- **printenv PATH** : valeur d'une variable précise
- **unset MA_VAR** : supprime la variable

Fichiers de configuration du shell

- **Globaux** : `/etc/profile` (login) et `/etc/bash.bashrc` (non-login) — tous les utilisateurs
- **Personnels** : `~/.bash_profile`, `~/.profile`, `~/.bashrc`, `~/.bash_logout`
- **Ordre des fichiers de login** : `.bash_profile` puis `.bash_login` puis `.profile` — le premier trouvé est lu
- `~/.bashrc` : personnalisation des shells interactifs — alias, export PATH, PS1
- `/etc/skel/` : fichiers copiés dans le \$HOME des nouveaux comptes
- **Vérifier** : `source ~/.bashrc` recharge la configuration sans se reconnecter

Shell de login vs non-login

- **Shell de login** (connexion console/SSH) : `/etc/profile` → `~/.bash_profile/~/.profile` → `~/.bashrc`
- **Shell non-login** (terminal graphique) : `/etc/bash.bashrc` → `~/.bashrc`
- **Script** : aucun fichier de configuration
- **.bash_profile** source souvent `~/.bashrc` pour uniformiser
- **Annotation** : « après édition de `~/.bashrc` : source `~/.bashrc` »

Section 9 : Archivage et compression

Archiver vs compresser → tar → gzip, bzip2, xz, zip → exemple de sauvegarde

Archiver vs compresser

- **Archiver** : regrouper plusieurs fichiers en UN SEUL (tar) — sans toucher au contenu
- **Compresser** : réduire la taille des données (gzip, bzip2, xz)
- Ce sont DEUX opérations distinctes, souvent combinées
- **tar** seul → archive volumineuse ; **tar** + compression → archive compacte
- Extensions : **.tar**, **.tar.gz** / **.tgz**, **.tar.bz2**, **.tar.xz**, **.zip**
- **Ordre** : on archive puis on compresse (ou directement **tar czf**)

tar : créer, lister, extraire

- **tar cf archive.tar fichiers** : crée une archive (**c** = create, **f** = file)
- **tar xf archive.tar** : extrait (**x** = extract) ; **tar tf archive.tar** : liste (**t** = list)
- **-v** : mode verbeux ; **-p** : préserve les permissions ; **-C dossier** : dossier cible
- Compression combinée : **tar czf** (gzip), **tar cjf** (bzip2), **tar cJf** (xz)
- Extraire : **tar xzf archive.tgz** — les chemins du dépôt guident la restauration
- Bonne pratique : **tar tf** avant d'extraire une archive inconnue

gzip, bzip2, xz et zip

- **gzip fichier** : compresse → **fichier.gz** (le plus rapide) ; **gunzip** décompresse
- **bzip2 fichier** : meilleur taux que gzip → **.bz2** ; **bunzip2**
- **xz fichier** : le meilleur taux, le plus lent → **.xz** ; **unxz**
- Comparatif : gzip rapide/faible taux, bzip2 intermédiaire, xz lent/fort taux
- **zip** / **unzip** : format compatible Windows, archive + compression intégrées
- **gzip -c** : sortie sur stdout — utilisable avec **>** ou un pipe

Exemple : sauvegarde avec tar

- `tar czf backup-$(date +%F).tar.gz ~/projets` : sauvegarde datée du jour
- `$(date +%F)` produit `2026-07-31` — substitution de commande
- `tar tjf archive.tar.bz2` : liste le contenu sans extraire
- `tar xzf backup-2026-07-31.tar.gz -C /tmp/restaure` : restauration ciblée
- `gzip -d fichier.gz` équivaut à `gunzip fichier.gz`
- **Vérifier** : `ls -lh backup-*.tar.gz`

Section 10 : Localisation des fichiers

find : principes → critères et actions → which, type, whereis → exemple comparatif

find : principes

- `find [chemin] [expression]` : parcourt l'arborescence en temps réel
- Par nom : `find /etc -name "*.pdf"`
- Le chemin peut être `/` (tout le système), `.` (répertoire courant), un dossier précis
- find est précis mais lent : il lit réellement le disque
- Sans droits sur un répertoire : message « Permission denied » (à ignorer)
- Les motifs entre guillemets pour éviter l'expansion prématurée du shell

find : critères et actions

- `find /home -name "*.txt" -size +1M` : fichiers texte volumineux
- `find . -type f -mtime -7` : fichiers modifiés depuis moins de 7 jours
- `-type f` fichier · `-type d` répertoire · `-type l` lien
- `-size +1M / -size -10k` : tailles · `-mtime +30` : fichiers anciens · `-user ilias` : propriétaire
- **Actions** : `-print` (défaut) · `-delete` · `-exec commande {} \;`
- **Avec pipes** : `find . -name "*.log" | wc -l`

locate et updatedb

- **locate nom** : recherche dans une BASE DE DONNÉES préconstruite des noms de fichiers
- Beaucoup plus rapide que find, mais résultats potentiellement obsolètes
- La base est mise à jour périodiquement par **updatedb** (tâche cron)
- **updatedb** (en root) actualise manuellement
- **locate bashrc** retourne toutes les occurrences
- Piège : fichiers récents absents, fichiers supprimés encore listés
- **Installation**: `sudo apt update && sudo apt install plocate`

which, type, whereis

- **which commande** : chemin complet de la première occurrence dans le PATH
- **type commande** : comment le shell interprète (builtin, alias, fichier)
- **whereis commande** : binaire + pages man + sources
- **Exemple** : **which ls** → /usr/bin/ls • **type ls** → ls is aliased to 'ls --color=auto'
- **which** : utile dans les scripts pour vérifier qu'un outil est installé
- Rappel : type était déjà vu avec les commandes internes/externes

Exemple : find vs locate

- `find /etc -name "*.conf" -type f` : configuration actuelle, précise
- `locate "*.conf"` : même idée via la base, instantané
- `find . -type f -mtime -7` : fichiers récents du répertoire courant
- `find /home -name "*.txt" -size +1M -exec ls -lh {} \;` : détail des gros fichiers
- **Pipeline** : `find /etc -type f | wc -l` : compter les fichiers de /etc
- Au TP2 : combiner find, cut, sort, wc pour analyser le système

Quiz 3 : Environnement, archivage et localisation

- **Q1. Quelle commande rend une variable visible par les processus enfants ?**
 - A. echo • B. env • C. export • D. unset
- **Q2. Quel fichier de configuration est lu par un shell interactif NON-login (terminal graphique) ?**
 - A. /etc/profile • B. ~/.bash_profile • C. ~/.bashrc • D. ~/.bash_logout
- **Q3. tar czf backup.tar.gz dossier permet de...**
 - A. extraire une archive • B. archiver et comprimer avec gzip • C. lister une archive •
 - D. chiffrer un dossier
- **Q4. Quelle commande de recherche est très rapide mais peut renvoyer des résultats obsolètes ?**
 - A. find • B. whereis • C. locate • D. cat

Résumé du chapitre 2

- **Shell** : interpréteur de commandes, internes/externes, historique, complétion, man
- **Navigation** : chemins absolus/relatifs, pwd, cd, ls, dot files, arborescence FHS
- **Fichiers** : types, touch, cp, mv, rm, mkdir, rmdir, métacaractères (globbing)
- **Contenu** : cat, less, head, tail, wc, sort, uniq, cut
- **Redirections et pipes** : stdin/stdout/stderr, > >> < 2> &> 2>&1, |, tee
- **Environnement** : variables, PATH, export, ~/.bashrc, shells de login/non-login
- **Archivage et localisation** : tar, gzip/bzip2/xz, zip ; find, locate, which, type, whereis